

Name: M. Chethan Krishna
Reg No: 23BAI0093
Speech and Language Processing LAB DA - 7

Voice-Enabled Chatbot using Speech Recognition and Deep Learning

Live application	https://voicebot-slp.vercel.app
Source code	https://github.com/Chethan616/slp-proj
Trained model	https://huggingface.co/Chethan616/voice-chatbot-intent-distilbert

1. Objective

The task is to build and publicly deploy a chatbot that accepts spoken input, converts that speech to text using a speech recognition technique, classifies the resulting text with a deep learning model, and returns an appropriate response. Both the recognised speech and the generated reply must be visible to the user.

The finished system is a two-stage pipeline. Recorded audio is transcribed by Whisper, and the transcript is classified into one of 16 food and nutrition intents by a fine-tuned DistilBERT model, and the answer is looked up in a corpus of 39,447 real recipes. The predicted intent selects the response. Both stages run on the server.

```
microphone -> recorded audio -> POST /api/voice
              Whisper base.en (int8, CTranslate2) -> recognised text
              DistilBERT classifier (int8 ONNX, 16) -> intent + confidence
              recipe corpus lookup -> reply
```

2. Dataset

2.1 Source

The classifier is trained on CLINC150 (Larson et al., EMNLP 2019), a benchmark created specifically for intent classification. It contains 150 in-scope intents across ten domains with 150 crowdsourced utterances each, and - unusually for an intent dataset - an explicit out-of-scope class of queries a task-oriented assistant is not built to answer.

2.2 Subset

One CLINC domain is used in full - kitchen and dining, 15 intents - plus the out-of-scope class, giving 16 classes. The selection is fixed in `train/prepare_data.py` and is therefore reproducible. Taking a whole domain rather than a slice of several gives the assistant a subject it can genuinely answer about, and makes the out-of-scope class more important rather than less: a narrow assistant is asked things it cannot handle far more often than a general one.

Split	Utterances	Per in-scope intent	Out-of-scope
Train	1,750	100	250
Validation	375	20	75
Test	600	30	150

The out-of-scope examples in the test split were subsampled from the 1,000 CLINC provides. Left at full size, 1,000 out-of-scope utterances against 1,200 in-scope ones would have let a single class dominate every aggregate metric.

2.3 Second dataset: the recipe corpus

CLINC150 is a classification dataset and contains no replies, so a second corpus supplies the answers. `recipes-with-nutrition` (datahiveai, Hugging Face) holds 39,447 recipes with servings, energy, full nutrient breakdowns, ingredient lists and dietary labels.

It cannot train an intent classifier, since it contains recipes rather than user utterances, but it is the knowledge base the food intents answer from. Asked how many calories are in butter chicken, the system classifies the intent with DistilBERT, matches the dish against the 39,447 recipe names, and quotes the real figure. Where no dish is named, or where the corpus holds nothing relevant, a written template is used instead.

Dish matching is lexical rather than learned: query tokens are scored against recipe names with inverse document frequency weighting and normalised by name length, so a short exact title outranks a long one that merely shares a word. A multi-word query must match on more than one token, which is what stops an unrelated request finding a recipe.

The published file is 450 MB of nested JSON. A preparation script reduces it to the fields the assistant quotes and writes 2.4 MB gzipped, small enough to deploy beside the models.

3. Speech Recognition

Speech recognition uses Whisper (Radford et al., 2022) through the faster-whisper implementation, which runs the model on the CTranslate2 inference engine. The `base.en` checkpoint is used: 74 million parameters and English-only, which is both faster and more accurate on English than the multilingual checkpoint of the same size.

Audio is resampled to 16 kHz and converted into an 80-channel log-Mel spectrogram. The short-time Fourier transform gives the energy at each frequency in each short time window, the Mel scale warps the frequency axis to match human pitch perception, and the logarithm compresses the dynamic range. That representation is passed to a transformer encoder, and a transformer decoder

autoregressively emits text tokens while attending to the encoder output. Whisper was trained on 680,000 hours of weakly supervised audio, which is why it generalises to unfamiliar microphones without any adaptation here.

Two settings reduce cost. Weights are stored with int8 quantisation instead of 32-bit floats, roughly quartering memory use. Decoding uses greedy search rather than beam search, which approximately halves latency with no measurable accuracy cost on utterances this short.

4. Model Architecture

Four classifiers were trained on identical splits so that the contribution of the transformer could be measured rather than assumed. All are implemented in PyTorch; scikit-learn provides the classical baseline. Only the last is deployed.

- TF-IDF with logistic regression. Unigram and bigram features with sublinear term frequency scaling, fed to a multinomial logistic regression. No neural network, and therefore the reference point for whether deep learning earns its place.
- Bag-of-words with a multilayer perceptron. A binary bag-of-words vector into two hidden layers of 256 and 128 units, ReLU activations and dropout of 0.5.
- Embedding with a bidirectional LSTM. A learned 128-dimensional embedding, a BiLSTM with 64 hidden units per direction, masked mean-pooling over real tokens, dropout, then a linear classifier. Unlike the first two, this model can use word order.
- DistilBERT, fine-tuned. A six-layer distillation of BERT-base, about 40 percent smaller and 60 percent faster while retaining roughly 97 percent of BERT performance on GLUE. A linear classification head is attached over the [CLS] representation and the entire network is fine-tuned with cross-entropy loss.

Input utterance

```
-> WordPiece tokenisation, max length 32
-> DistilBERT encoder: 6 transformer layers, hidden 768, 12 heads
-> [CLS] representation (768-d)
-> pre-classifier Linear(768, 768) + ReLU + dropout(0.2)
-> Linear(768, 16) -> softmax over 16 classes
```

Trainable parameters: 66,965,776.

Hyperparameter	Value
Checkpoint	distilbert-base-uncased
Max sequence length	32
Batch size	32
Epochs	5
Optimiser	AdamW, learning rate 3e-5
Schedule	10 percent linear warmup, then linear decay

Hyperparameter	Value
Weight decay	0.01
Gradient clipping	1.0
Model selection	best validation macro F1

5. Methodology

- Data preparation. CLINC150 is downloaded, the 40 chosen intents are extracted, out-of-scope examples are subsampled with a fixed seed, and the splits are written to disk. Train, validation and test come from the dataset own splits, so no utterance appears in more than one.
- Training. All four models are trained on the training split. The neural models keep the checkpoint with the best validation macro F1 rather than the final epoch.
- Evaluation. Every model is scored once on the held-out test split. Predictions are saved, so all figures and tables derive from a single set of numbers.
- Threshold selection. The confidence threshold for rejecting an utterance as out-of-scope is chosen by sweeping it on the validation split and taking the value that maximises macro F1. It is only then measured on test. Choosing it on test would leak test information into a deployed hyperparameter.
- Export. The trained model is exported to ONNX and quantised to int8 for deployment, and the quantised graph is checked against the full-precision one before it is used.
- End-to-end evaluation. Held-out test utterances are synthesised to speech with offline system voices and run through the complete deployed pipeline, giving a word error rate and the intent accuracy actually achieved from audio.

All randomness is seeded, so the reported numbers reproduce.

5.1 Out-of-scope handling

A classifier with a softmax output must assign every input to some class. Asked something outside its training distribution it will answer confidently and wrongly. Two mechanisms guard against this. First, out-of-scope is a trained class, so the model can predict it directly. Second, if the maximum softmax probability falls below a threshold, the prediction is overridden to out-of-scope regardless of the argmax, and the reply states that the question was not understood.

6. Results

6.1 Intent classification

All models were evaluated on the same 600 held-out utterances.

Model	Parameters	Test accuracy	Macro F1
TF-IDF + logistic regression	112,864	0.8750	0.8803
Bag-of-words + MLP	249,744	0.8867	0.8847
Embedding + BiLSTM	209,040	0.8867	0.8763
DistilBERT, fine-tuned	66,965,776	0.9333	0.9294
DistilBERT, int8 ONNX (deployed)	-	0.9333	0.9278

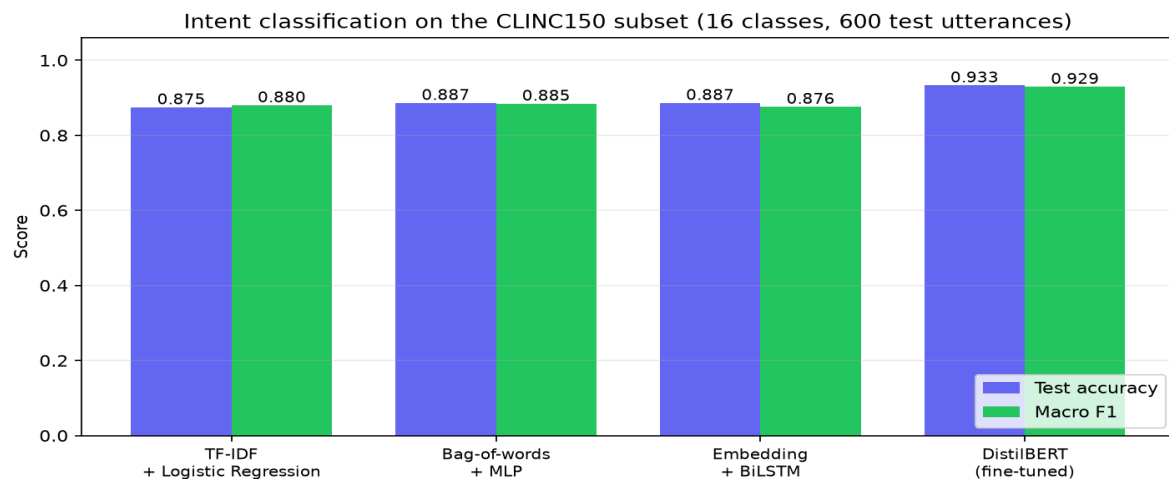


Figure 1. Accuracy and macro F1 across the four classifiers.

The fine-tuned transformer improves on the best non-transformer model by 4.7 accuracy points and 4.5 macro-F1 points. The gain comes from pre-training: DistilBERT has already learned that phrasings such as "what is the forecast" and "will it rain tomorrow" are related, whereas the from-scratch models can only learn that from the hundred examples per class they are given.

The test set holds 150 out-of-scope utterances against 30 for each in-scope intent, because a narrow assistant is asked out-of-scope questions far more often than a general one. Accuracy and macro F1 are both reported for that reason: the first is dominated by the largest class, the second weights every class equally.

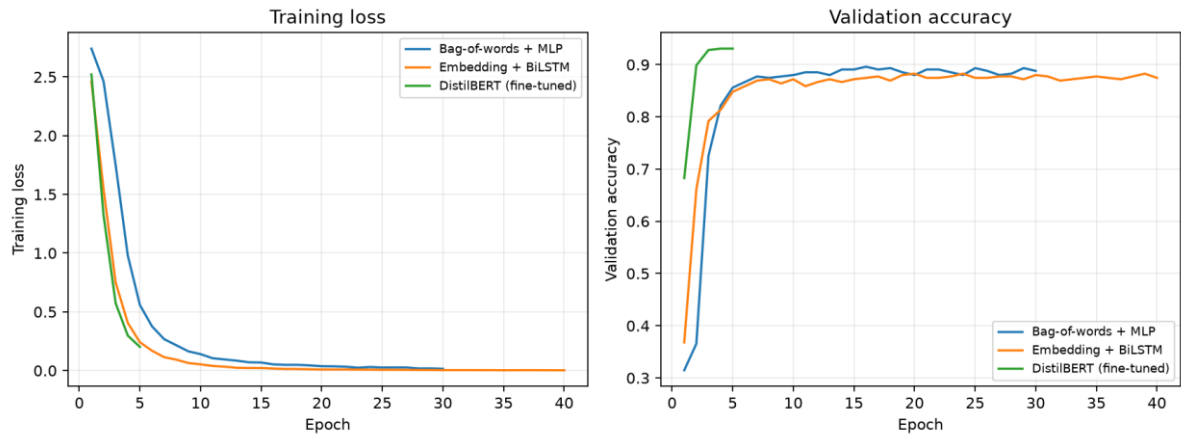


Figure 2. Training loss and validation accuracy per epoch.

6.2 Out-of-scope rejection

Setting	In-scope accuracy	Out-of-scope recall	Macro F1
Test, no threshold	0.9356	0.9267	0.9294
Validation, threshold 0.38	0.9333	0.9467	0.9390
Test, threshold 0.38	0.9222	0.9467	0.9273

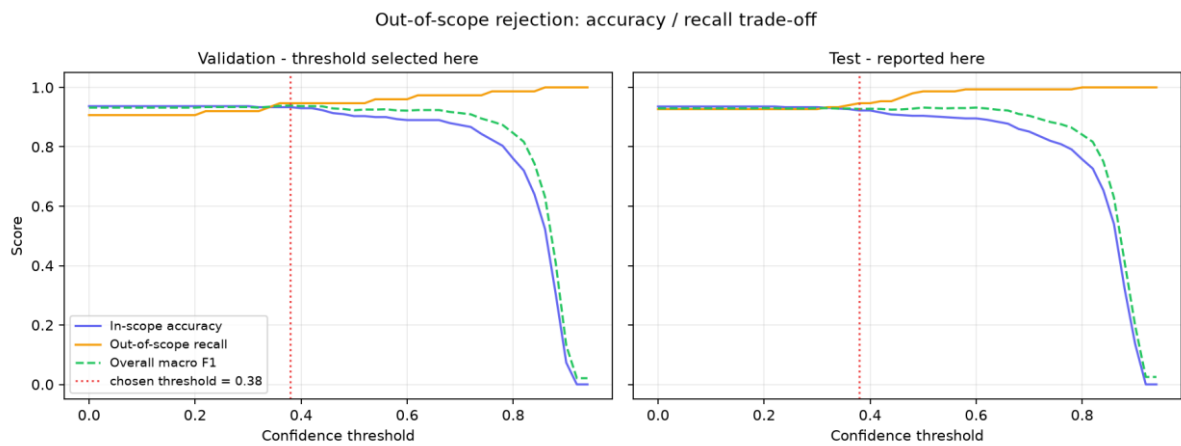


Figure 3. The confidence threshold swept on validation, where it is chosen, and on test, where it is reported.

The threshold raises out-of-scope recall on test from 92.7 percent to 94.7 percent at a cost of 1.3 points of in-scope accuracy. A higher threshold catches more unanswerable questions but begins refusing questions the model actually got right.

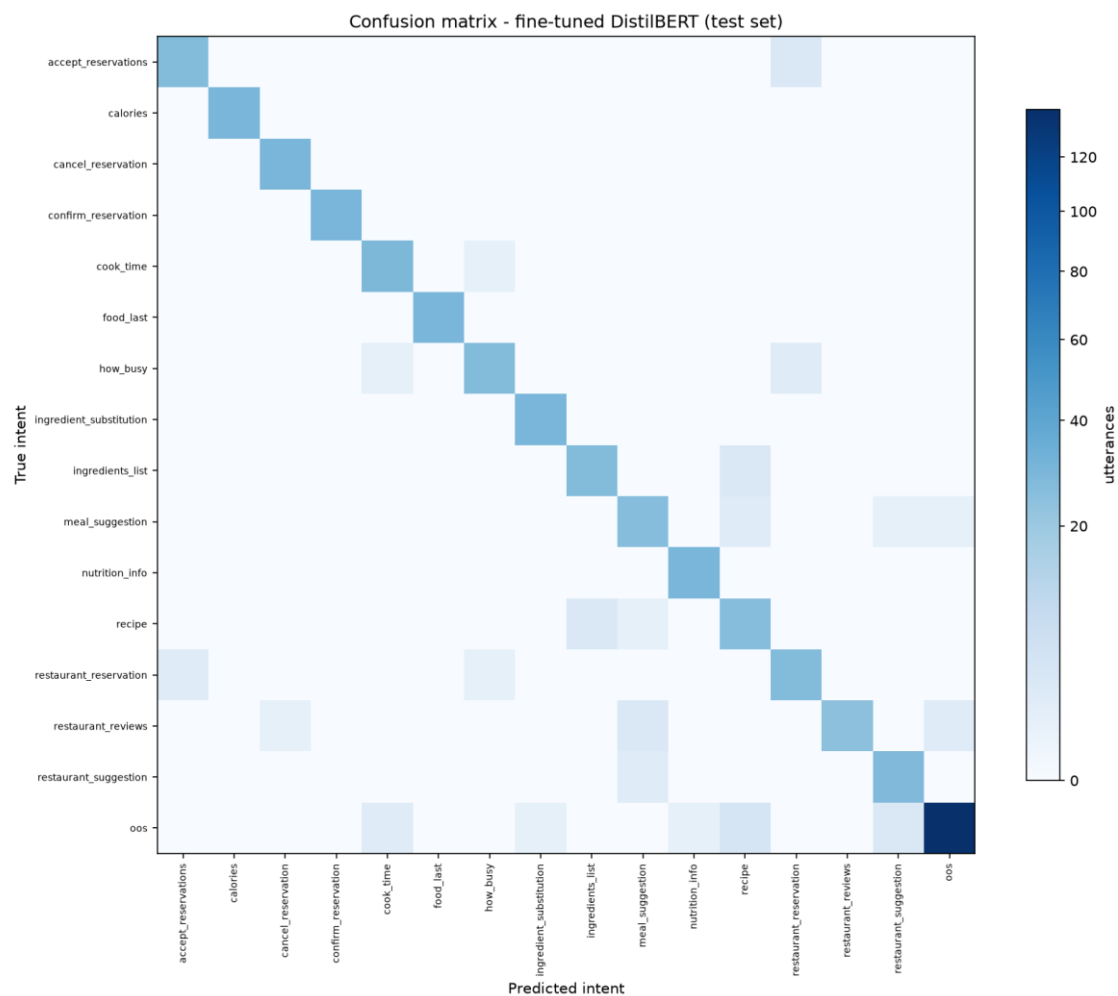


Figure 4. Confusion matrix for the deployed model on the test split.

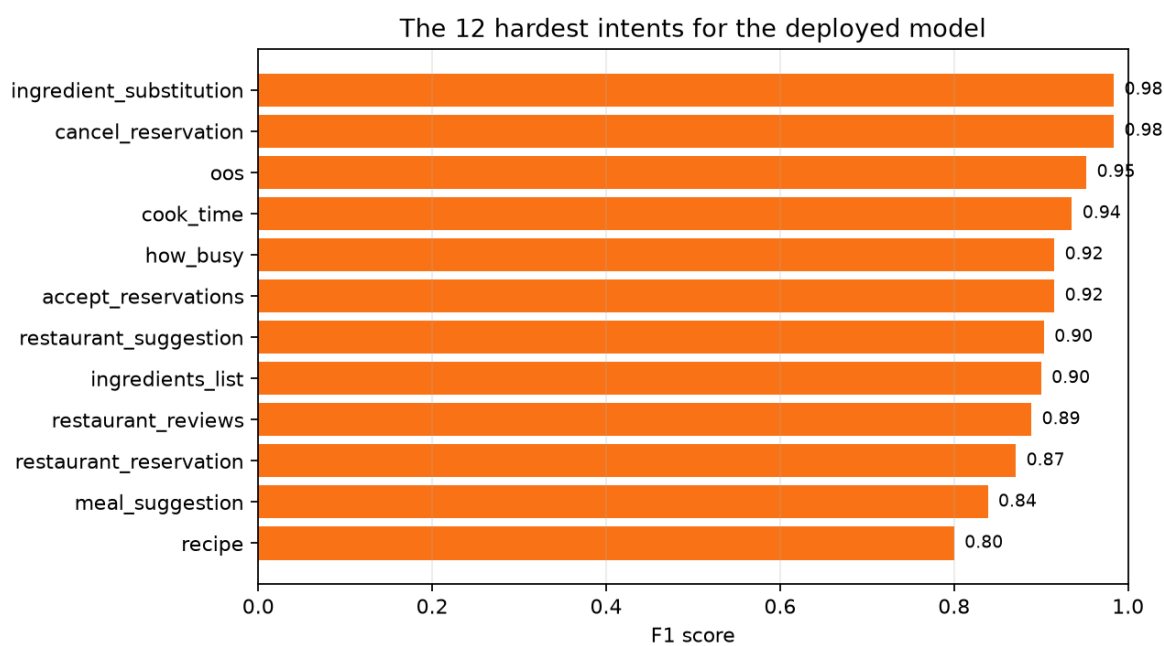


Figure 5. The intents the model finds hardest.

6.3 End-to-end through speech

Text accuracy is not the accuracy a user experiences, because recognition errors propagate into the classifier. To measure that, 36 held-out test utterances were synthesised to speech and put through the complete deployed pipeline.

Metric	Value
Utterances evaluated	36
Word error rate	0.0476
Transcribed with no errors	75.0 percent
Intent accuracy from clean text	1.0000
Intent accuracy from speech	0.9722
Accuracy lost to recognition	2.8 points
Mean speech recognition latency	2,515 ms
Mean intent inference latency	12 ms
Real-time factor	0.74

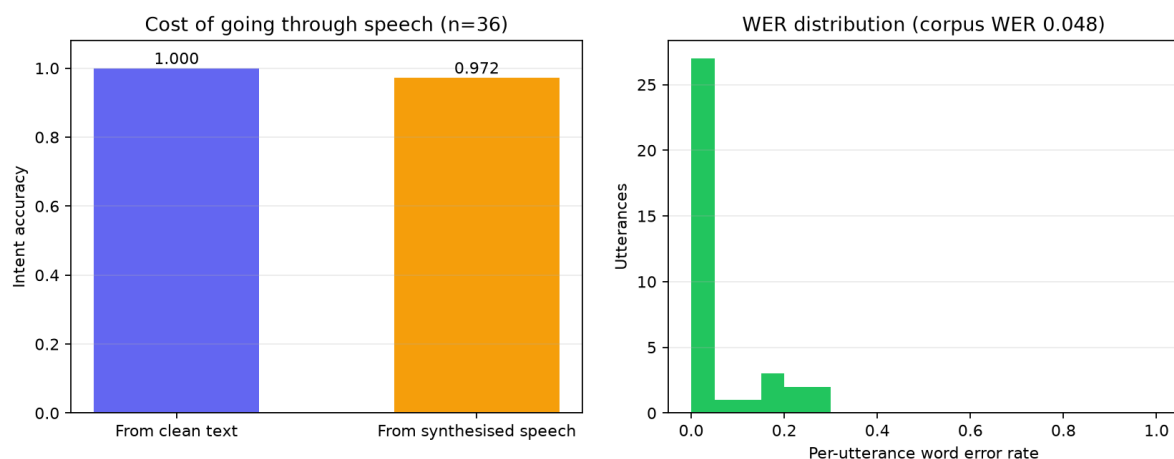


Figure 6. Intent accuracy from speech against clean text, and the distribution of word error rate.

A word error rate of 0.048 means roughly five words in every hundred are recognised wrongly, and 75 percent of utterances come back with no errors at all. Intent accuracy falls by only 2.8 points when the input arrives as speech rather than text, because recognition errors tend to land on words that do not determine the intent. A real-time factor of 0.74 means the system transcribes faster than the audio was spoken.

This evaluation uses synthetic speech, which is cleaner than real speech: no background noise, no disfluencies and limited accent variation. The figures are a lower bound on error, not a prediction of field performance.

7. Deployment

The backend is a FastAPI service in a Docker container, deployed on Railway. The front end is a static build served by Vercel and calls the API over CORS. Both models are loaded once at startup and reused; loading them per request would dominate the latency budget.

PyTorch is a training-time dependency only. Whisper runs on CTranslate2, and the classifier was exported to an int8-quantised ONNX graph served by ONNX Runtime. This matters on a free hosting tier, where the default Linux PyTorch wheel would pull in roughly 2.5 GB of CUDA libraries that a CPU host never executes.

Property	PyTorch build	Deployed ONNX int8
Model size on disk	268.0 MB	67.4 MB
Test accuracy	0.9333	0.9333
Macro F1	0.9294	0.9278
Relative inference speed	1.0x	1.9x

Quantisation is lossless within noise here. The int8 graph agrees with the full-precision model on 97.9 percent of test utterances, and where they disagree the errors cancel, leaving accuracy marginally higher rather than lower. The end-to-end figures in section 6.3 were measured on this deployed build.

Measured on the deployed container: about 360 MB under load, roughly 12 seconds from cold start to serving, and a few seconds per voice request.

MealMuse Website

Production site: <https://voicebot-fawn.vercel.app>

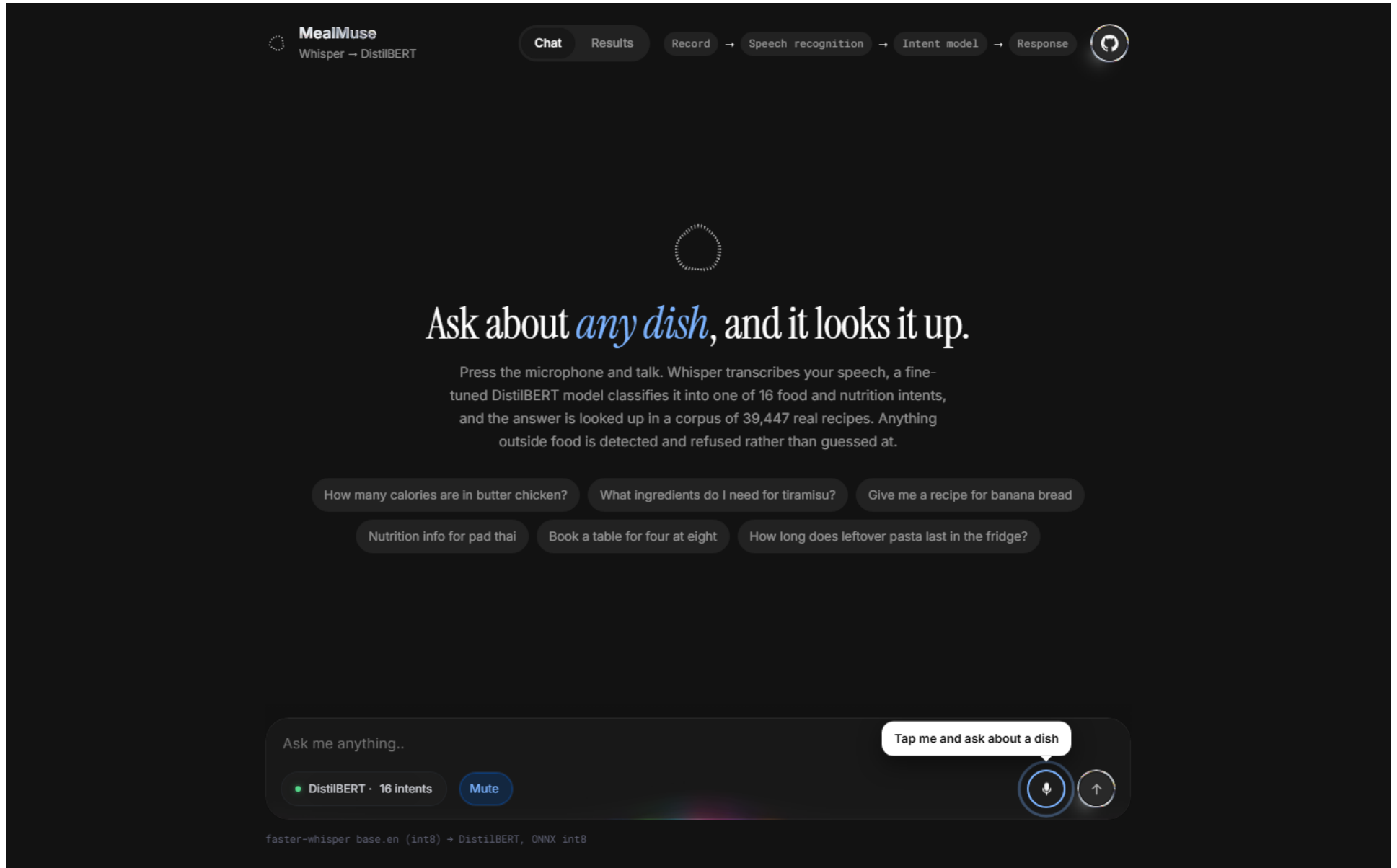


Figure. MealMuse website interface.